

Module 10: Pointers & Dynamic Memory

Introduction to C++ Programming — Part 2: Intermediate C++

Module Overview

This is the first module of Part 2, and the concept most students arrive dreading. Frame it from the start as a generalization of something they already trust: pass-by-reference (Part 1, Module 7) and the Part 1 capstone's narrow `Student*` / `nullptr` pattern. The goal is a working mental model of 'a pointer stores an address' plus disciplined `new/delete` hygiene - not memory-model trivia.

Suggested Duration: 75–90 minutes

Learning Objectives

By the end of this module, students will be able to:

- Explain what a pointer stores (an address) and use `&` and `*` correctly.
- Check a pointer against `nullptr` before dereferencing it.
- Allocate a runtime-sized array with `new[]` and free it with `delete[]`.
- Name and recognize the three classic pointer bugs: memory leak, dangling pointer, double free.
- Explain the relationship between arrays and pointers.

Prerequisites

- All of Part 1, especially Module 7 (pass by reference) and the Module 9 capstone's `Student*` pattern.

Materials & Instructor Prep

- A whiteboard/slide with a simple memory diagram (boxes with addresses) - this module benefits enormously from a visual, not just code.
- 10-pointers-dynamic-memory/examples/ preloaded and ready to run in sequence.

Key Vocabulary

Term	Definition
Address	The numeric location in memory where a variable's value is stored.
Pointer	A variable that stores an address instead of a value directly.
& (address-of)	Operator that returns the address where a variable lives.
* (dereference)	Operator that, given an address, accesses the value stored there.
nullptr	A special pointer value meaning 'points to nothing.'
Heap / dynamic memory	Memory allocated at runtime with new, sized however the program needs at that moment, and freed explicitly with delete.
Memory leak	Memory allocated with new but never freed with delete - permanently unusable for the rest of the program's life.
Dangling pointer	A pointer still holding an address whose memory has already been freed.

Lesson Outline & Instructor Talking Points

1. The mental model, with a diagram, before any code (15 min)

- Draw two boxes on the board: a variable box labeled 'age = 25' at some address (e.g. 0x1000), and a second box labeled 'agePtr' whose contents is literally '0x1000' - the pointer doesn't hold 25, it holds the ADDRESS of the box that holds 25.
- Only after the diagram lands, introduce & ('address-of', reads the address off the first box) and * ('dereference', follows the address to get the value).

Instructor note: *This diagram is worth more than any code example in this module. Do not skip straight to syntax - students who get the boxes-and-arrows picture first rarely struggle with the syntax after.*

2. Live demo: & and * together (15 min)

- Run examples/01_address_and_dereference.cpp live. Print age, &age, agePtr, and *agePtr side by side so students see &age and agePtr are literally the same value.
- Then write through the pointer (*agePtr = 30;) and show age itself changed - this is the 'aha' moment: a pointer isn't a copy, it's a way to reach the original.

Instructor note: *Explicitly connect back to Module 7: 'This is exactly what a reference parameter was doing for you automatically - a reference is a pointer with safety rails built in.'*

3. nullptr and the habit of checking (10 min)

- Run examples/02_nullptr.cpp. Ask: 'what happens if we dereference p before checking?' before running the crash - let them predict the danger, not just read about it.
- Tie directly back to the Part 1 capstone's Student* findStudent(...) returning nullptr - name it explicitly as the same pattern, now formalized.

4. new[] / delete[] - dynamic memory (25 min)

- Motivate directly: 'Every array size you've used so far had to be known when you WROTE the code. What if you don't know how many readings there will be until the program is RUNNING?'
- Live-code examples/03_dynamic_array.cpp, reading n from the user and allocating exactly that many doubles.
- State the rule plainly and put it on the board: every new[] needs exactly one matching delete[]. Set the pointer to nullptr immediately after deleting it - make this a physical habit, not just a rule to remember.

Instructor note: *Live-demo a memory leak by removing the delete[] line and explaining (without necessarily needing special tooling) that the memory is now unreachable and wasted for the rest of the run - the CONCEPT matters more than proving it with a profiler at this level.*

5. The three classic bugs, named explicitly (15 min)

- Put all three on the board with one-line definitions: memory leak (forgot to free), dangling pointer (used after freeing), double free (freed twice).
- Live-demonstrate a dangling pointer/double-free scenario from the README, and note that delete nullptr; is explicitly safe - which is exactly why setting a pointer to nullptr after deleting it is a real defensive habit, not superstition.

6. Pointers and arrays (10 min)

- Run examples/04_pointer_array_relationship.cpp. Keep this brief and framed as 'recognize this when you see it elsewhere' rather than a skill to drill - vector and [] indexing remain the default tools going forward.

Common Student Misconceptions

Misconception	How to Address It
"A pointer and the variable it points to are basically the same thing."	A pointer is a separate variable that happens to store an address - draw the two-box diagram again if this comes up.

Misconception	How to Address It
"Dereferencing a null pointer usually just gives you 0 or an empty value."	It's undefined behavior - typically an immediate crash, not a safe default. Demonstrate live if your environment allows it.
"Once I call delete, the pointer becomes automatically null."	delete frees the memory but leaves the pointer holding the same (now invalid) address - you must set it to nullptr yourself.

Exercises & Module Project

Exercise: 10-pointers-dynamic-memory/exercises/exercise1.cpp - address/dereference basics plus a dynamically-allocated, user-filled array. Project (three tracks in project/README.md): Generic - Dynamic Grade Book; EE - ADC Sample Buffer; Biomedical - Vital Sign Buffer. All three practice the identical skill (new[]/delete[] plus min/max/average) in a different domain.

Facilitation guidance: Let students self-select a track based on their own field rather than assigning one - the point is relevance, not uniformity. Whichever track a student picks, grade the SAME core skill: correct new[]/delete[] pairing, no leaks, correct statistics. The domain framing is motivation, not a different rubric.

Discussion & Comprehension Check Questions

1. How is a pointer similar to, and different from, a reference parameter from Module 7?

Guidance: Similar: both let code reach across a function boundary and touch the original variable, not a copy. Different: a reference can never be null or reseated; a raw pointer can be either, which is more flexible and more dangerous.

2. Why must delete[] be called exactly once per new[]?

Guidance: Fewer than once leaks the memory permanently; more than once (double free) is undefined behavior, often a crash.

Assessment Ideas

- Whiteboard trace: given a short sequence of pointer operations (declare, assign, dereference, delete), have students draw the box-and-arrow diagram at each step.
- Bug hunt: provide a snippet with a missing delete[] or a use-after-delete, and ask students to name which of the three classic bugs it is.

Answer Key Notes

See [10-pointers-dynamic-memory/exercises/solutions/exercise1_solution.cpp](#).

Wrap-Up & Bridge to Next Module

Bridge forward: "Today you learned to manage ONE object's data through a pointer. Next module, we use this same idea to build families of related classes that share data and behavior through inheritance."

Instructor Tips

- This module has the highest anxiety-to-difficulty ratio in the course. Slow, visual, diagram-first pacing pays off disproportionately here compared to almost any other module.
- Resist introducing smart pointers (`unique_ptr`/`shared_ptr`) as a 'better way' mid-lesson - it undercuts the point of learning the underlying mechanics first. Mention them only as a clearly-labeled 'next step after this course.'